

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores
Licenciatura en Ingeniería en Computadores

CE-1113 Sistemas Empotrados

Documento Técnico y de Diseño (DI)

Proyecto II

Co-diseño Hardware/Software de SoC
para Reproductor de Audio Avanzado en DE1-SoC

Integrantes

Tenorio Cervantes, Javier Francisco	javsrec02@estudiantec.cr
Flores Lara, Axel Bruno	axeel@estudiantec.cr
Marín Muñoz, Kendall Jossue	kendallmarin@estudiantec.cr
Varela Venegas, Julio Josué	juliojvv20@estudiantec.cr

Profesor: Dr.-Ing. Jeferson González Gómez
Fecha de entrega: 11 de mayo de 2026
Enlace al Repositorio: Repositorio GitHub

Índice

1	Introducción	2
1.1	Contexto del proyecto	2
1.2	Objetivo del documento	2
2	Identificación de necesidades y requerimientos (DI1)	3
2.1	Descripción del problema	3
2.2	Requerimientos funcionales	3
2.3	Requerimientos no funcionales	3
2.4	Restricciones y consideraciones	4
3	Análisis de alternativas de solución (DI2)	4
3.1	Decisiones de partición hardware/software	4
3.1.1	Filtros de audio digitales: FPGA vs HPS	4
3.1.2	Control de almacenamiento SD: FPGA vs HPS	5
3.1.3	Control temporal	5
3.1.4	LCD 16x2: PIO en FPGA vs GPIO del HPS	5
3.1.5	Tabla resumen de partición	6
4	Diseño de la solución seleccionada (DI3)	7
4.1	Arquitectura general del sistema	7
4.2	Diseño de hardware en Platform Designer	8
4.2.1	Sistema en chip generado (Qsys)	8
4.2.2	Módulo de hardware personalizado	9
4.2.3	Mapa de memoria del sistema	11
4.2.4	Interfaz de audio: AUDIO_IF_0	12
4.2.5	PIO de botones con captura de flancos	13
4.3	Diseño de software	14
4.3.1	Arquitectura modular	14
4.3.2	Inicialización y mapeo de memoria	15
4.3.3	Pipeline de reproducción PCM	16
4.3.4	Filtros de audio digitales	16
4.3.5	Extracción de metadatos WAV	17
4.3.6	Arranque automático del sistema	17
4.4	Consideraciones sobre el uso del HPS vs Nios II	18
5	Validación del diseño (DI4)	19
5.1	Cumplimiento de requerimientos	19
5.2	Pruebas realizadas	19
5.2.1	Arranque automático del sistema	19
5.2.2	Pruebas de filtros de audio	20
5.2.3	Pruebas de metadatos WAV	22
5.2.4	Prueba de estabilidad	22
5.2.5	Prueba de la LCD HD44780	23
5.3	Trade-offs observados	24
6	Conclusiones	24
7	Referencias	25

1. Introducción

1.1. Contexto del proyecto

El co-diseño hardware/software es una de las metodologías centrales en el desarrollo de sistemas embebidos modernos. A diferencia de los enfoques puramente secuenciales donde primero se diseña el hardware y luego el software, el co-diseño explora en paralelo el espacio de partición de funcionalidades entre la lógica programable (FPGA) y el procesador (HPS), buscando un balance óptimo entre rendimiento, consumo de recursos y complejidad de implementación.

Este proyecto consistió en diseñar e implementar un **reproductor de audio portable avanzado** sobre la plataforma **Altera DE1-SoC**, utilizando el procesador **ARM Cortex-A9** disponible como Hard Processing System (HPS) y la FPGA Cyclone V. El sistema integra periféricos generados con **Platform Designer** (Qsys), lógica personalizada en FPGA, y una aplicación en C ejecutada sobre Linux embebido mínimo desde el HPS, con acceso directo al hardware sin capas de abstracción.

La Figura 1 muestra el sistema completo ensamblado: la tarjeta DE1-SoC con la pantalla LCD 16x2 conectada mediante cables Dupont al header GPIO_0, y el módulo de amplificación de audio con parlante de 8Ω conectado a la salida de línea del códec WM873. (Este último módulo mencionado no se encontraba conectado a la hora de tomar la foto, presentada en la Figura 1).

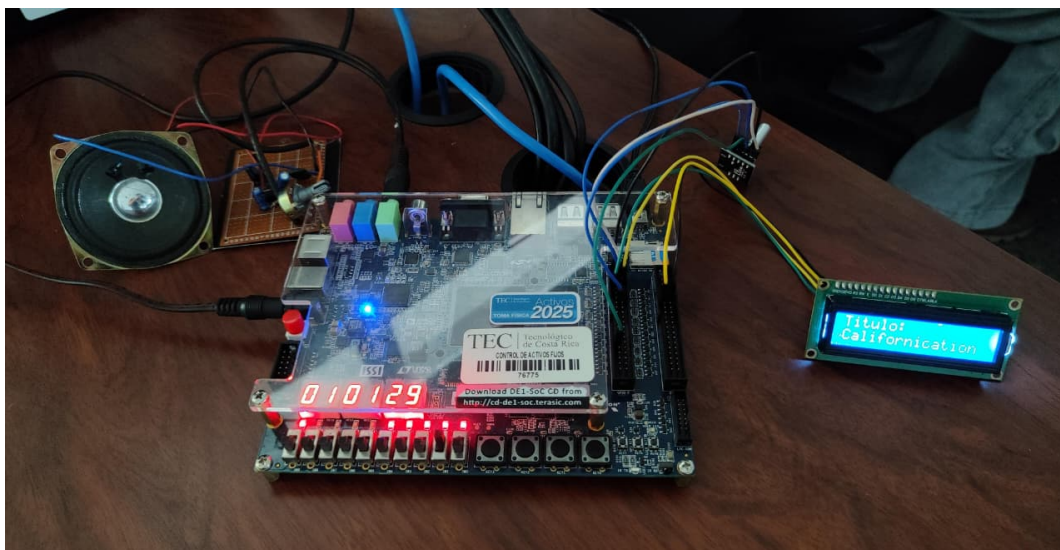


Figura 1: Sistema completo montado: DE1-SoC con LCD 16x2

1.2. Objetivo del documento

Este documento evidencia el proceso de diseño seguido durante el desarrollo del proyecto, cubriendo la identificación de requerimientos, el análisis de alternativas de partición hardware/software, el diseño detallado de la solución y la validación del sistema implementado. Se documentan las decisiones arquitecturales tomadas y los trade-offs evaluados, siguiendo los

indicadores del atributo profesional **Diseño (DI1–DI4)**.

2. Identificación de necesidades y requerimientos (DI1)

2.1. Descripción del problema

Se requiere implementar un reproductor de audio portable completo sobre la DE1-SoC, capaz de reproducir archivos en formato WAV (16-bit, frecuencias estándar), con control físico mediante botones y switches, visualización en displays de 7 segmentos y pantalla LCD 16x2, y procesamiento digital de audio mediante filtros seleccionables en tiempo real.

El sistema debe demostrar el dominio del co-diseño hardware/software, accediendo directamente al hardware sin capas de abstracción (HAL), y justificando explícitamente qué componentes se implementan en FPGA versus en el procesador HPS.

2.2. Requerimientos funcionales

- Reproducción de archivos de audio en formato WAV/PCM desde tarjeta SD, con soporte para al menos 10 canciones.
- Controles físicos de reproducción: Play/Pausa, Stop, Siguiente y Anterior, mediante los botones KEY0–KEY3 de la DE1-SoC.
- Visualización del tiempo de reproducción en formato MM:SS sobre los displays de 7 segmentos HEX0–HEX3, y número de canción actual en HEX4–HEX5.
- Mínimo 3 filtros de audio digitales seleccionables mediante switches físicos con aplicación en tiempo real y latencia menor a 100 ms.
- Extracción y visualización de metadatos de las canciones (título, artista, álbum, duración) en pantalla LCD 16x2 y consola serial.
- Reproducción continua automática al finalizar cada canción.
- Indicación visual del filtro activo mediante LEDs.
- Visualización del nivel de amplitud de la señal de audio (peakmeter) en LEDs.
- Control de volumen mediante encoder rotativo mapeado al códec WM8731.

2.3. Requerimientos no funcionales

- Tiempo de inicio del sistema inferior a 10 segundos tras el reset.
- Respuesta a botones inferior a 200 ms.
- Transición entre canciones en menos de 2 segundos.
- Reproducción continua y estable sin interrupciones ni reinicios inesperados.

- Acceso directo al hardware mediante registros mapeados a memoria, sin uso de HAL.
- Sistema operativo mínimo en el HPS, sin interfaz gráfica local.
- Gestión de repositorio con Git Flow y Conventional Commits.

2.4. Restricciones y consideraciones

- **Plataforma:** Altera DE1-SoC con ARM Cortex-A9 (HPS) y FPGA Cyclone V.
- **Herramienta de diseño HW:** Platform Designer (Qsys) de Altera, Quartus Prime 18.1 Lite.
- **Procesador principal:** ARM Cortex-A9 en el HPS. No se utilizó NIOS II soft core; la justificación se detalla en la Sección 4.4.
- **Compilación:** SoC EDS 18.1 con GCC para ARM/Linux, compilación cruzada.
- **Formato de audio:** Los archivos WAV se convierten a PCM crudo mediante FFmpeg (32 kHz, estéreo, 16-bit signed little-endian).
- **Interfaz de audio:** el manejo del hardware de audio se realiza exclusivamente desde la FPGA mediante el IP de audio en Platform Designer.
- **Sin HAL:** el acceso a periféricos se hace directamente mediante `mmap()` sobre `/dev/mem`, sin las funciones del HAL de Altera.

3. Análisis de alternativas de solución (DI2)

3.1. Decisiones de partición hardware/software

El co-diseño obligó a tomar decisiones explícitas sobre qué componentes implementar en la FPGA y cuáles en el HPS. Esta sección analiza las principales decisiones y justifica el porqué de cada una.

3.1.1. Filtros de audio digitales: FPGA vs HPS

Alternativa 1 — Filtros en FPGA (hardware DSP). Implementar los filtros como módulos RTL en la FPGA, procesando las muestras antes de enviarlas al DAC. Ventajas: latencia mínima garantizada, sin carga al procesador, paralelismo natural. Desventajas: complejidad de diseño en HDL, dificultad para modificar coeficientes en tiempo de ejecución, requiere recompilación del bitstream para cambios.

Alternativa 2 — Filtros en HPS (software DSP). Implementar los filtros en C, procesando cada muestra PCM antes de escribirla al FIFO del IP de audio. Ventajas: facilidad de implementación, acceso nativo a punto flotante, depuración sencilla. Desventajas: carga adicional al procesador, latencia dependiente del scheduler.

Decisión adoptada: Filtros en software (HPS). A 32 kHz, el procesador tiene $\sim 31 \mu s$ por muestra, holgado para el ARM Cortex-A9 a 800 MHz. El filtro pasa-banda biquad IIR usa aritmética de punto flotante, lo que habría sido significativamente más complejo en HDL. La latencia de cambio de filtro resultó imperceptible en la práctica, bien por debajo del límite de 100 ms.

3.1.2. Control de almacenamiento SD: FPGA vs HPS

Se optó por gestionar el acceso a la tarjeta SD **desde el HPS mediante software**, aprovechando el driver SDIO integrado de Linux. Implementar el protocolo SD y un sistema de archivos FAT en HDL habría tenido un costo de desarrollo desproporcionado para los beneficios obtenidos.

3.1.3. Control temporal

El tiempo de reproducción se lleva mediante un contador de frames en software (`frames_played / PCM_SAMPLE_RATE`), sin timers de hardware dedicados. Esto mantiene sincronización exacta con el audio reproducido sin consumir recursos en la FPGA.

3.1.4. LCD 16x2: PIO en FPGA vs GPIO del HPS

Alternativa 1 — Módulo dedicado en Platform Designer. Agregar un módulo/-periférico dedicado para el control de la LCD dentro del sistema Platform Designer, conectándolo al bus LW-AXI para que pueda ser accedido desde el HPS mediante registros mapeados en memoria. Esta alternativa permite integrar la LCD de forma uniforme con el resto de periféricos del sistema, utilizando el mismo mecanismo de acceso MMIO empleado para botones, switches, displays y módulos de audio. Su principal desventaja es que requiere regenerar el HDL, actualizar el archivo `.qip`, recompilar el diseño en Quartus y programar nuevamente la FPGA con el archivo `.sof` resultante.

Alternativa 2 — GPIO del HPS directamente. Otra alternativa posible era controlar la LCD directamente desde los GPIO del HPS, mapeando los registros del banco GPIO mediante `mmap()` y generando las señales de control desde software. Esta opción evita modificar el diseño de hardware en FPGA, pero hace que la temporización de las señales dependa completamente del software y del sistema operativo, lo cual puede ser menos conveniente en un sistema donde también se ejecuta reproducción de audio en tiempo real.

Decisión adoptada: módulo dedicado en Platform Designer con acceso MMIO. Se seleccionó la primera alternativa, implementando un periférico dedicado para la LCD dentro del diseño de hardware. Este módulo fue conectado al bus LW-AXI y posteriormente accedido desde la aplicación en C ejecutada en Linux mediante direcciones mapeadas en memoria. Para incorporar este cambio fue necesario regenerar el HDL del sistema, actualizar los archivos de integración del proyecto, recompilar el diseño en Quartus y generar un nuevo archivo `.sof`. Esta decisión permitió mantener una arquitectura más ordenada y consistente con el resto de periféricos del sistema.

3.1.5. Tabla resumen de partición

Componente	En FPGA	En HPS	Decisión
Interfaz audio DA-C/ADC	Generación de señales y FIFO de audio con baja latencia	No recomendable por requerimientos temporales del códec	FPGA (IP)
Peakmeter / Encoder	Cálculo de nivel de señal, lectura del encoder y control de LEDs	Lectura de registros y ajuste del volumen en el códec	FPGA (IP) + HPS
Filtros DSP	Implementación posible, pero con mayor complejidad en HDL	Flexible, modificable y con rendimiento suficiente	HPS (SW)
Control SD / FAT	Implementación compleja en hardware	Soporte nativo mediante Linux y sistema de archivos	HPS (SW)
Displays 7 segmentos	PIOs de salida conectados a los displays	Cálculo del formato ##:MM:SS y escritura por MMIO	FPGA (PIO) + HPS
Botones / Switches	PIOs de entrada y captura de eventos físicos	Interpretación de eventos y lógica de control del reproductor	FPGA (PIO) + HPS
Control temporal	Posible mediante temporizadores hardware	Contador de frames PCM para tiempo de reproducción	HPS (SW)
LCD 16x2	PIO bidireccional conectado al LW-AXI para controlar SDA/SCL	Driver I2C por software y escritura de metadatos mediante MMIO	FPGA (PIO) + HPS
Configuración WM8731	IP OC I2C Master conectada al bus del sistema	Secuencia de configuración enviada desde software	FPGA (IP) + HPS

Cuadro 1: Resumen de decisiones de partición hardware/software

4. Diseño de la solución seleccionada (DI3)

4.1. Arquitectura general del sistema

La Figura 2 muestra el diagrama de arquitectura completo del sistema. Se distinguen claramente los dos dominios: el **ARM Cortex-A9 (HPS)** que ejecuta la lógica de la aplicación en Linux, y la **FPGA Cyclone V** que implementa en hardware los IPs de audio, peakmeter, encoder, GPIO y la interfaz I2C hacia el códec WM8731. La comunicación entre ambos dominios se realiza a través del bus **HPS-FPGA Bridge / MMIO (LW-AXI)**.

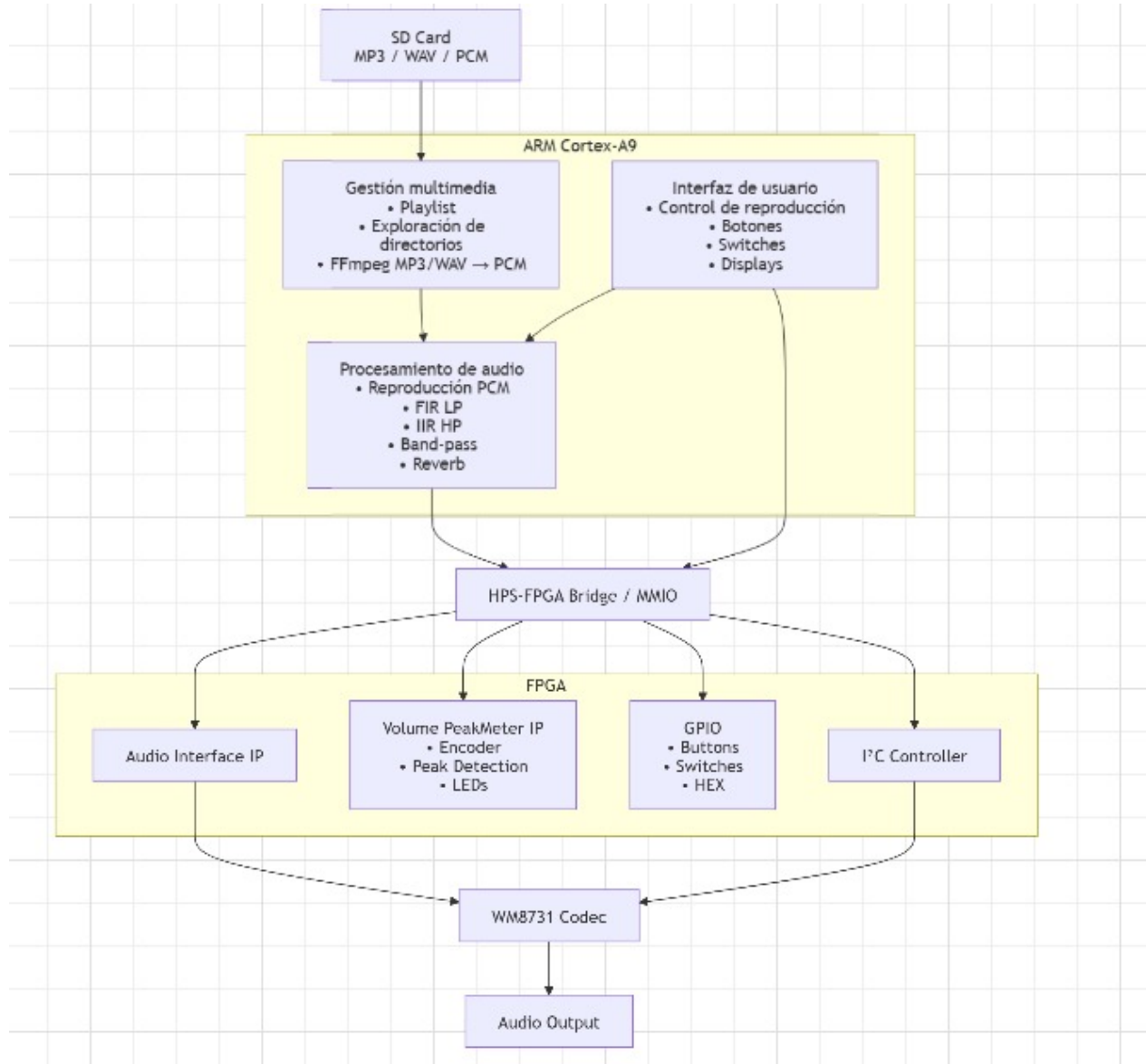


Figura 2: Diagrama de arquitectura del sistema: HPS (ARM Cortex-A9) y FPGA (Cyclone V) con sus respectivos módulos, comunicados a través del bridge HPS-FPGA/MMIO.

El sistema se organiza en tres capas:

- **Capa HPS/Linux:** Imagen Linux mínima sobre el ARM Cortex-A9. Ejecuta la aplicación del reproductor en C, que gestiona la playlist, convierte audio con FFmpeg, aplica filtros DSP por software y coordina todos los periféricos mediante acceso directo a registros vía mmap().
- **Capa FPGA / Platform Designer:** Sistema en chip con los IPs de audio (AUDIO_IF_0), I2C (oc_i2c_master_0), módulo personalizado de peakmeter y control de volumen, y PIOs para botones, switches, LEDs, displays de 7 segmentos y LCD.
- **Periféricos externos:** Tarjeta SD con las canciones, pantalla LCD 16x2 conectada mediante un backpack I2C controlado desde un PIO bidireccional en FPGA, códec de audio WM8731, salida de audio y periféricos físicos de interacción como botones, switches, LEDs, displays y encoder.

4.2. Diseño de hardware en Platform Designer

4.2.1. Sistema en chip generado (Qsys)

La Figura 3 muestra el diagrama del sistema generado en Platform Designer, con todos los componentes conectados al bus LW-AXI y sus señales de conduit hacia el top-level de Quartus (ghrd_top.v).

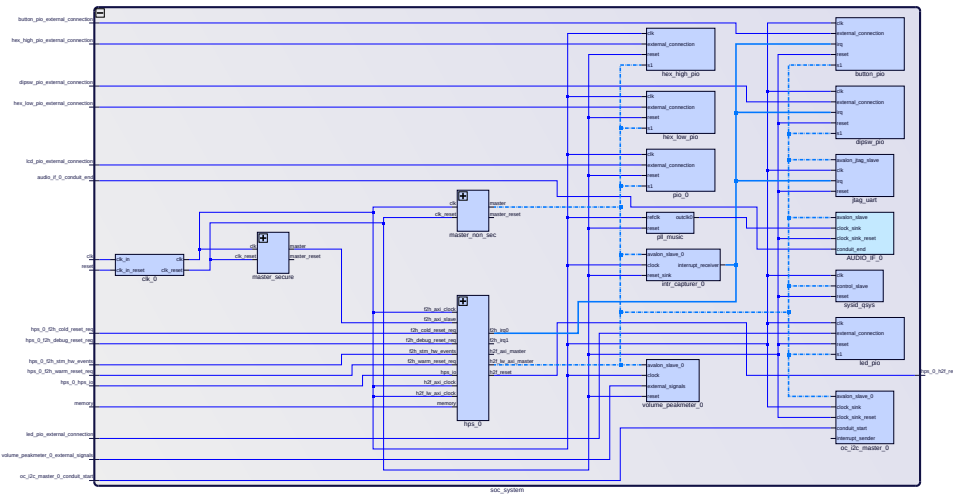


Figura 3: Diagrama del sistema generado en Platform Designer (Qsys): IPs conectados al bus LW-AXI del HPS, con sus interfaces Avalon-MM y señales de conduit.

Los componentes principales del sistema Qsys son:

- **AUDIO_IF_0:** IP de interfaz de audio. Implementa los FIFOs de entrada y salida del códec WM8731 (DAC y ADC), exponiéndolos como registros Avalon-MM accesibles desde el HPS.

- **oc_i2c_master_0:** Controlador I2C OpenCore. Permite al HPS configurar los registros internos del WM8731 (volumen, sample rate, formato) mediante el protocolo I2C de 2 hilos.
- **volume_peakmeter_0:** IP personalizado que recibe muestras de audio, detecta el nivel de amplitud pico y controla los LEDs de forma proporcional. También procesa las señales del encoder rotativo para ajuste de volumen.
- **pio_0 (LCD I2C):** PIO para la interfaz LCD.
- **button_pio:** PIO de entrada con captura de flancos descendentes e IRQ para los botones KEY[3:0].
- **dipsw_pio:** PIO de entrada con captura de flancos para los switches SW[9:0].
- **led_pio:** PIO de salida para los LEDs LEDR[9:0].
- **hex_low_pio / hex_high_pio:** PIOs de salida para los displays de 7 segmentos HEX0–HEX5.
- **intr_capturer_0:** Módulo de captura de interrupciones para diagnóstico.
- **sysid_qsys:** Identificador único del sistema para verificación de compatibilidad hardware/software.
- **jtag_uart:** UART sobre JTAG para depuración.
- **pll_music:** PLL que genera el reloj de audio requerido por el códec WM8731 (MCLK = 18.432 MHz para operación a 32 kHz).

4.2.2. Módulo de hardware personalizado

Como parte del sistema se implementó un módulo de hardware personalizado denominado `volume_peakmeter`. Este módulo fue integrado en Platform Designer como un periférico conectado al bus Avalon-MM/LW-AXI, permitiendo que la aplicación ejecutada en el HPS pueda interactuar con él mediante registros mapeados en memoria.

El propósito principal del módulo es visualizar la amplitud de la señal de audio reproducida mediante los LEDs de la tarjeta, de forma similar a los medidores de nivel presentes en reproductores de audio tradicionales. Además, el módulo permite leer el valor de volumen generado por un encoder, el cual posteriormente es utilizado por el software para configurar el volumen de salida del códec de audio WM8731.

Funcionalidad general El módulo recibe muestras de audio desde la aplicación en C y calcula una representación simplificada del nivel de amplitud de la señal. Esta información se refleja en los LEDs de la tarjeta, permitiendo observar visualmente los cambios de intensidad del audio en tiempo real. A mayor amplitud de la muestra recibida, mayor cantidad de LEDs se activan, simulando el comportamiento de un peakmeter o medidor de nivel.

Adicionalmente, el módulo expone un registro de volumen actualizado por el encoder. El software lee periódicamente este valor y lo traduce al rango de configuración aceptado por el códec WM8731, permitiendo modificar el volumen durante la ejecución del reproductor sin detener la reproducción.

Interfaz del módulo La comunicación entre el HPS y el módulo `volume_peakmeter` se realiza mediante registros mapeados en memoria. Estos registros permiten consultar el nivel actual del peakmeter, configurar el estado de reproducción, leer el valor del volumen y enviar muestras de audio al módulo.

Offset	Registro	Acceso	Descripción
0x00	STATUS	Lectura	Contiene el valor actual del peakmeter. Los bits menos significativos representan el nivel de amplitud visualizado mediante LEDs.
0x04	CTRL	Escritura	Permite indicar el estado actual de reproducción. El software escribe en este registro si el reproductor se encuentra en estado STOP, PLAY o PAUSE. Este estado puede utilizarse para controlar un LED indicador.
0x08	VOLUME	Lectura	Contiene el valor actual del volumen generado por el encoder. El valor leído por software se encuentra en un rango discreto y posteriormente se convierte al rango utilizado por el códec WM8731.
0x0C	AUDIO	Escritura	Registro utilizado para enviar una muestra de audio al módulo. El hardware utiliza esta muestra para calcular la amplitud y actualizar el nivel mostrado en el peakmeter.
0x10	BTN	Lectura/Escritura	Registro asociado al botón del encoder. Puede utilizarse para detectar eventos de presión y limpiar el evento desde software mediante escritura.

Cuadro 2: Interfaz de registros del módulo personalizado `volume_peakmeter`.

Mapa de bits del registro de control El registro CTRL permite comunicar al hardware el estado actual de reproducción. Este valor es escrito desde la aplicación en C cada vez que el reproductor cambia de estado.

Valor	Estado	Descripción
0x0	STOP	Indica que no existe reproducción activa. El indicador de estado permanece apagado.
0x1	PLAY	Indica que una canción se encuentra en reproducción. El indicador de estado puede parpadear para mostrar actividad.
0x2	PAUSE	Indica que la reproducción se encuentra pausada. El indicador de estado permanece encendido de forma fija.

Cuadro 3: Valores utilizados en el registro CTRL del módulo `volume_peakmeter`.

4.2.3. Mapa de memoria del sistema

La Figura 4 muestra el mapa de memoria completo del sistema, con todos los periféricos mapeados sobre el bus LW-AXI. El espacio total utilizado es de 288 B (rango 0x0000–0x011F), con 256 B de registros activos.

Mapa de memoria — 0x0000 a 0x011F

LW-AXI bridge

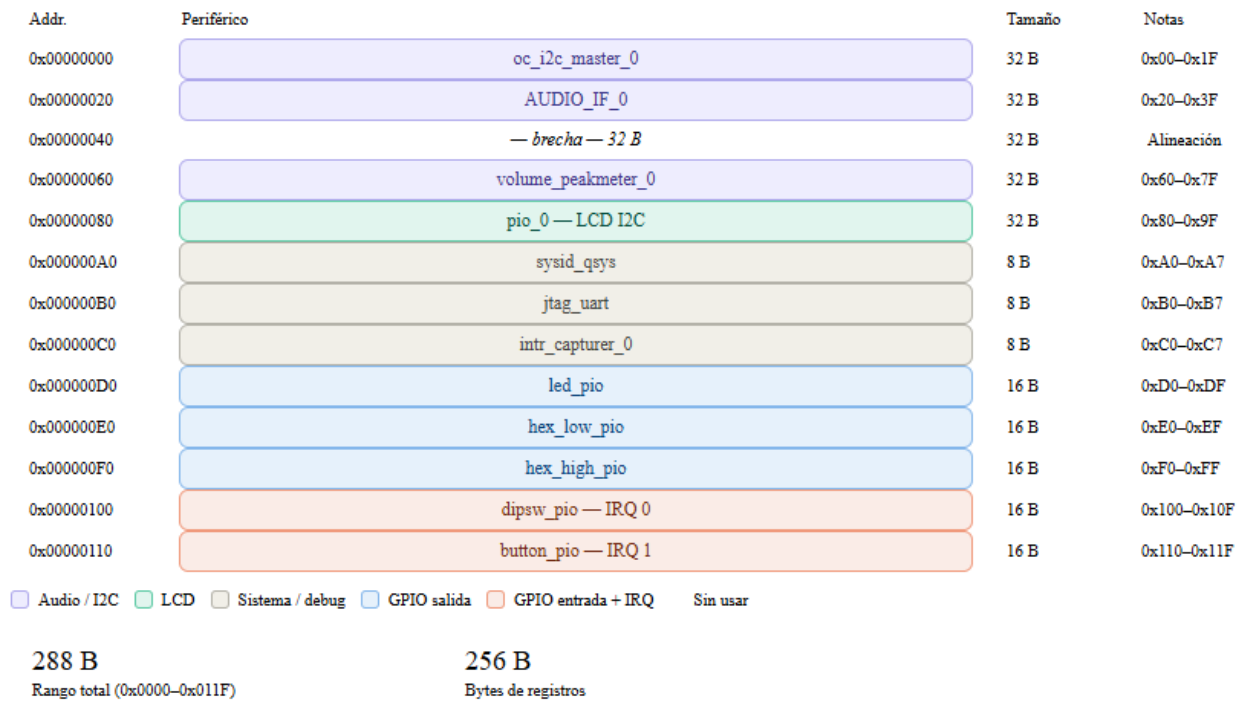


Figura 4: Mapa de memoria del sistema sobre el bus LW-AXI (offsets 0x0000–0x011F). Los colores indican la categoría funcional de cada periférico.

La dirección física absoluta de cada periférico se calcula como:

$$\text{dirección_física} = 0xFF200000 + \text{offset_mapa}$$

donde 0xFF200000 es la dirección base del espacio LW-AXI en el Cyclone V HPS. La tabla siguiente detalla los periféricos con sus offsets y funciones:

Componente	Offset base	Tamaño	Función
oc_i2c_master_0	0x0000	32 B	Configuración WM8731 vía I2C
AUDIO_IF_0	0x0020	32 B	FIFO DAC/ADC del códec de audio
volume_peakmeter	0x0060	32 B	Peakmeter y control de encoder
pio_0 (LCD I2C)	0x0080	32 B	PIO para interfaz LCD
sysid_qsys	0x00A0	8 B	Identificador del sistema
jtag_uart	0x00B0	8 B	UART de depuración por JTAG
intr_capturer_0	0x00C0	8 B	Captura de interrupciones
led_pio	0x00D0	16 B	LEDs LEDR[9:0]
hex_low_pio	0x00E0	16 B	Displays HEX0–HEX3 (28 bits)
hex_high_pio	0x00F0	16 B	Displays HEX4–HEX5 (14 bits)
dipsw_pio (IRQ 0)	0x0100	16 B	Switches SW[9:0] con IRQ
button_pio (IRQ 1)	0x0110	16 B	Botones KEY[3:0] con IRQ

Cuadro 4: Mapa de memoria detallado del sistema sobre LW-AXI

4.2.4. Interfaz de audio: AUDIO_IF_0

El componente AUDIO_IF_0 forma parte de la infraestructura esencial del sistema de audio. Su función es permitir la comunicación entre la lógica del sistema y el códec WM8731, exponiendo una interfaz para enviar muestras PCM hacia la FIFO del DAC. Este componente no se considera el módulo personalizado principal del proyecto, sino una IP base necesaria para la reproducción de audio.

Durante la ejecución, la aplicación en C escribe muestras estéreo de 16 bits en la interfaz de audio. Posteriormente, estas muestras son enviadas al códec para generar la salida analógica. En conjunto con el módulo volume_peakmeter, esta interfaz permite tanto la reproducción del audio como la visualización de su amplitud mediante LEDs. Implementa los FIFOs de comunicación con el códec WM8731 y expone la siguiente interfaz de registros:

Offset	Registro	Acceso	Descripción
+0	DAC_LFIFO	Escritura	Muestra canal izquierdo al DAC (16-bit)
+1	DAC_RFIFO	Escritura	Muestra canal derecho al DAC (16-bit)
+2	ADC_LFIFO	Lectura	Captura canal izquierdo del ADC
+3	ADC_RFIFO	Lectura	Captura canal derecho del ADC
+4	CMD	Escritura	bit[0]=1: limpiar ambos FIFOs
+5	STATUS	Lectura	bit[0]=DAC FIFO lleno; bit[1]=ADC FIFO vacío

Cuadro 5: Interfaz de registros del módulo AUDIO_IF_0

El acceso se realiza sin HAL, directamente sobre el puntero `audio_addr` obtenido del `mmap()`:

```

1 /* Verificar que el FIFO no est  lleno antes de escribir */
2 int AUDIO_DacFifoNotFull(void)
3 {
4     return (alt_read_word(audio_addr + AUDIO_STATUS_PORT)
5             & MASK_STATUS_DAC_FULL) ? 0 : 1;
6 }
7
8 /* Escribir una muestra est reo al FIFO del DAC */
9 void AUDIO_DacFifoSetData(int ch_left, int ch_right)
10 {
11     alt_write_word(audio_addr + AUDIO_DAC_LFIFO_PORT, ch_left & 0xFFFF);
12     alt_write_word(audio_addr + AUDIO_DAC_RFIFO_PORT, ch_right & 0xFFFF);
13 }

```

Listing 1: Escritura de muestra estéreo al DAC y verificación de FIFO

4.2.5. PIO de botones con captura de flancos

El `button_pio` se configuró con captura de flanco descendente (`EDGE_TYPE = FALLING`) e IRQ por flanco (`IRQ_TYPE = EDGE`). La detección de pulsaciones se hace mediante polling del registro de captura de flancos, limpiando los bits leídos inmediatamente para evitar eventos duplicados:

```

1 uint32_t buttons_get_events(void)
2 {
3     uint32_t edge = alt_read_word(button_pio_addr + PIO_EDGE_CAPTURE_REG) & 0
4     xF;
5     if (edge != 0)
6         alt_write_word(button_pio_addr + PIO_EDGE_CAPTURE_REG, edge);
7     return edge;
8 }

```

Listing 2: Lectura y limpieza del registro de captura de flancos

4.3. Diseño de software

4.3.1. Arquitectura modular

La aplicación está organizada en módulos con responsabilidad única. Cada módulo corresponde a un par de archivos `.h/.c` y encapsula completamente el acceso a su periférico o funcionalidad:

Módulo	Responsabilidad
<code>main.c</code>	Inicialización de hardware, mapeo de memoria, gestión de playlist y loop principal de navegación entre canciones
<code>audio_control</code>	Configuración del WM8731 vía OC I2C (volumen, sample rate, formato I2S), control de FIFOs DAC/ADC
<code>pcm</code>	Loop de reproducción frame a frame, integración de filtros DSP, extracción de metadatos WAV, manejo de eventos de botones
<code>audio_filter</code>	Implementación en software de los 4 filtros DSP: FIR pasa-bajos, IIR pasa-altos, reverberación, biquad pasa-banda
<code>button_control</code>	Inicialización del PIO de botones (habilitación de IRQ, limpieza de captura) y lectura de eventos
<code>switch_control</code>	Lectura del PIO de switches y traducción a tipo de filtro activo
<code>display_control</code>	Codificación de dígitos en 7 segmentos (activo en bajo) y escritura en HEX PIOs
<code>lcd_i2c</code>	Driver para LCD 16x2 con controlador HD44780 mediante backpack I2C. Utiliza un PIO bidireccional accedido por MMIO desde el HPS para controlar las líneas SDA y SCL.
<code>led_control</code>	Control de LEDs indicadores: filtro activo (LEDR0) y estado de reproducción (LEDR9)
<code>volume_peakmeter</code>	Control del módulo personalizado de peakmeter, lectura del volumen desde encoder, envío de muestras de audio para visualización en LEDs y actualización del estado de reproducción (PLAY, PAUSE, STOP)

Cuadro 6: Módulos de software y sus responsabilidades

A continuación, en la Figura 5 se presenta el diagrama de organización de software del sistema completo.

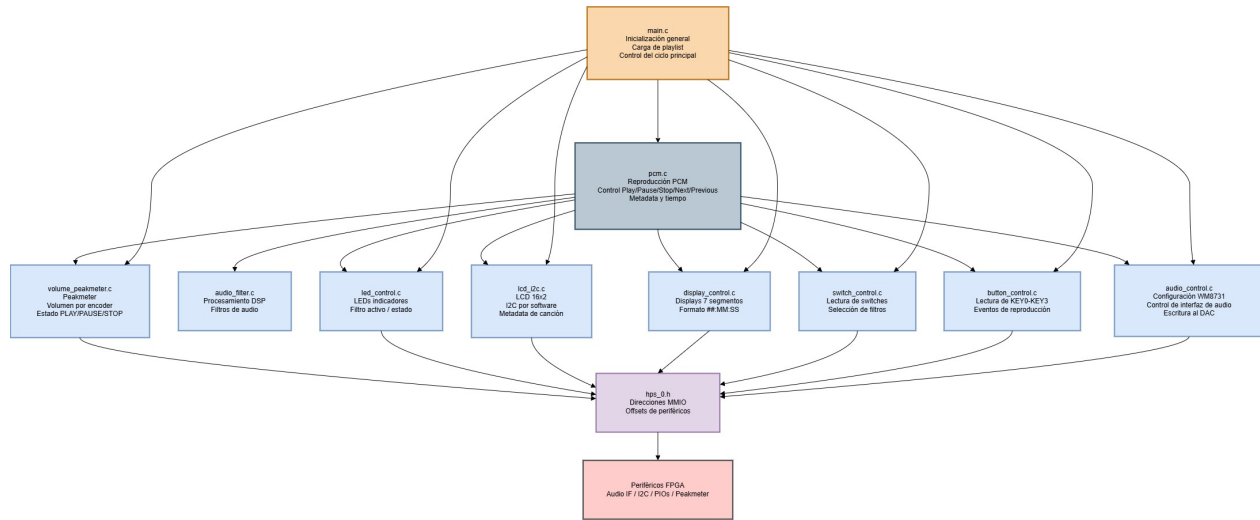


Figura 5: Diagrama de organización de software del sistema completo

El archivo `main.c` se encarga de inicializar el sistema, realizar el mapeo de los periféricos mediante memoria mapeada y cargar la playlist de canciones. La lógica de reproducción se concentra en `pcm.c`, el cual coordina la lectura del archivo PCM, el envío de muestras al DAC, la aplicación de filtros, el manejo de botones y la actualización de los periféricos de salida, como los displays de 7 segmentos, la LCD, los LEDs y el peakmeter.

Los módulos restantes funcionan como controladores especializados para cada periférico o funcionalidad del sistema. De esta forma, el software se organiza de manera modular, separando la lógica principal de reproducción de los drivers encargados del acceso a hardware.

4.3.2. Inicialización y mapeo de memoria

Todo el acceso a los periféricos implementados en FPGA se realiza mediante `mmap()` sobre `/dev/mem`. La aplicación mapea el espacio de memoria correspondiente al lightweight HPS-to-FPGA bridge y, a partir de esta dirección virtual base, calcula las direcciones de cada periférico utilizando los offsets generados en `hps_0.h`.

```

1
2 static volatile unsigned long *h2p_lw_axi_addr = NULL;
3 volatile unsigned long *oc_i2c_audio_addr = NULL;
4 volatile unsigned long *audio_addr = NULL;
5 volatile unsigned long *button_pio_addr = NULL;
6
7 /* Mapeo del espacio de memoria usado para acceder a los periféricos */
8 virtual_base = mmap(NULL, HW_REGS_SPAN,
9                     PROT_READ | PROT_WRITE,
10                    MAP_SHARED,
11                    fd,
12                    HW_REGS_BASE); /* ALT_STM_OFST */
13

```

```
14 );
```

Listing 3: Mapeo del espacio LW-AXI y acceso a periféricos

Los punteros individuales a cada periférico se derivan del puntero base sumando el offset del dispositivo enmascarado con HW_REGS_MASK:

```
1  h2p_lw_axi_addr = virtual_base +
2      ((unsigned long) (ALT_LWFPGASLVS_OFST) &
3      (unsigned long) (HW_REGS_MASK));
4
5  oc_i2c_audio_addr = virtual_base +
6      ((unsigned long) (ALT_LWFPGASLVS_OFST + OC_I2C_MASTER_0_BASE) &
7      (unsigned long) (HW_REGS_MASK));
8
9  audio_addr = virtual_base +
10      ((unsigned long) (ALT_LWFPGASLVS_OFST + AUDIO_IF_0_BASE) &
11      (unsigned long) (HW_REGS_MASK));
12
13  button_pio_addr = virtual_base +
14      ((unsigned long) (ALT_LWFPGASLVS_OFST + BUTTON_PIO_BASE) &
15      (unsigned long) (HW_REGS_MASK));
```

Listing 4: Derivación de punteros a periféricos individuales

4.3.3. Pipeline de reproducción PCM

Por cada frame de audio (4 bytes: 2 por canal), el loop en `play_PCM()` ejecuta:

1. Verificar eventos de botones (play/pausa, stop, siguiente, anterior).
2. Cada 1024 frames: leer switches y actualizar el filtro activo si cambió, reiniciando el estado interno del filtro anterior.
3. Leer 4 bytes del archivo PCM desde disco (`fread`).
4. Decodificar las muestras como `int16_t` signed little-endian.
5. Aplicar el filtro DSP activo: `audio_filter_process_stereo(&L, &R)`.
6. Esperar hasta que el FIFO del DAC tenga espacio disponible.
7. Escribir las muestras procesadas al FIFO: `AUDIO_DacFifoSetData(L, R)`.
8. Actualizar el contador de tiempo y refrescar los displays cada segundo.

4.3.4. Filtros de audio digitales

Se implementaron 4 filtros en `audio_filter.c`, seleccionables mediante SW0–SW3:

Switch	Filtro	Tipo	Parámetros clave
Ninguno	Bypass	Sin procesamiento	Muestra pasa sin modificar
SW3	Pasa-bajos FIR	Promedio móvil, 5 taps	Ventana rectangular uniforme
SW2	Pasa-altos IIR	Primer orden	$\alpha = 950/1024 \approx 0,928$
SW0	Reverberación	Buffer circular	Delay 150 ms (4800 muestras), feedback = 0.35
SW1	Pasa-banda IIR	Biquad 2do orden	$f_0 = 1 \text{ kHz}$, $Q = 0,8$, $f_s = 32 \text{ kHz}$

Cuadro 7: Filtros de audio implementados en software (HPS)

La verificación de cambio de filtro se hace cada 1024 frames para no introducir latencia perceptible. El filtro pasa-banda implementa la ecuación de diferencias de un biquad de segundo orden:

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2]$$

con coeficientes: $b_0 = 0,1087$, $b_1 = 0$, $b_2 = -0,1087$, $a_1 = -1,7484$, $a_2 = 0,7826$.

4.3.5. Extracción de metadatos WAV

El parser de metadatos en `pcm.c` reconstruye el path del archivo WAV original a partir del `.pcm` (quitando la extensión `.pcm` final) y extrae:

- **Header WAV estándar** (44 bytes): sample rate, canales, bits por muestra y tamaño del chunk data para calcular la duración exacta.
- **Chunk LIST/INFO**: sub-chunks INAM (título), IART (artista), IPRD (álbum), si existen.
- **Fallback por nombre**: si no hay metadata embebida, se parsea el nombre del archivo en formato `Artista - Título.wav`.

Los metadatos se imprimen en consola serial (PuTTY) y se muestran en la LCD al inicio de cada canción: título en la fila 0 y artista en la fila 1, truncados a 16 caracteres. Además en cuanto la canción se encuentra pausada, en la LCD se muestra el resto de metadatos asociados al archivo en tiempos de 300ms.

4.3.6. Arranque automático del sistema

La FPGA se programa automáticamente desde U-Boot al encender la tarjeta, cargando el archivo `soc_system.rbf` desde la partición FAT de la SD. Una vez que Linux inicia, el script `/etc/init.d/myplayer` (enlazado desde `/etc/rc5.d/S91myplayer`) ejecuta automáticamente el reproductor con la carpeta de canciones como argumento, sin intervención manual de forma `./MyPlayer songs`.

4.4. Consideraciones sobre el uso del HPS vs Nios II

La especificación permitía utilizar Nios II como soft core alternativo al HPS. Sin embargo, para este proyecto se optó por utilizar el **HPS (ARM Cortex-A9)** debido a las necesidades de manejo de archivos, procesamiento de audio y depuración del sistema.

- Linux embebido proporciona un sistema de archivos completo para acceder a la tarjeta SD, cargar canciones desde una carpeta y ejecutar FFmpeg para convertir archivos de audio a formato PCM. Esto evita implementar manualmente un sistema de archivos FAT o rutinas complejas de decodificación en bare-metal.
- El ARM Cortex-A9 ofrece mayor capacidad de procesamiento para tareas en software, como lectura de archivos, control de reproducción, cálculo de tiempo, manejo de metadatos y aplicación de filtros de audio. Además, su soporte de FPU facilita la implementación de filtros DSP como biquads IIR en software.
- El entorno Linux permite utilizar herramientas de depuración más completas, como `printf` mediante consola serial o SSH, `gdb`, utilidades de inspección del sistema y comandos estándar de Linux. Esto simplifica la validación del sistema durante la integración hardware/software.
- El uso de `mmap()` sobre `/dev/mem` permite acceder a los periféricos implementados en FPGA mediante direcciones MMIO, sin requerir un RTOS dedicado ni una pila de drivers compleja. Esto mantiene una arquitectura simple para la comunicación entre el software del HPS y los módulos de Platform Designer.

Aunque Nios II es adecuado para tareas de control embebido más simples o altamente acopladas a la FPGA, en este proyecto el HPS resultó más conveniente debido a la necesidad de manejar archivos de audio, ejecutar herramientas externas, depurar desde Linux y controlar múltiples periféricos mediante memoria mapeada.

5. Validación del diseño (DI4)

5.1. Cumplimiento de requerimientos

Requerimiento	Estado	Observación
Reproducción WAV/PCM desde SD	Cumplido	FFmpeg convierte al primer uso
Control Play/Pausa/Stop/Siguiente/Anterior	Cumplido	KEY0-KEY3, captura de flancos
Displays 7-seg MM:SS y número de canción	Cumplido	HEX0-HEX5, 1 actualización/s
Mínimo 3 filtros en tiempo real (<100 ms)	Cumplido	4 filtros implementados
Metadatos en consola serial	Cumplido	Título, artista, álbum, duración
Metadatos en LCD 16x2	Cumplido	Título fila 0, artista fila 1
Reproducción continua automática	Cumplido	Pasa automáticamente a la siguiente
LED indicador de filtro activo	Cumplido	LEDR0 refleja estado del filtro
Peakmeter en LEDs	Cumplido	LEDR[9:1] proporcional a amplitud
Control de volumen por encoder	Cumplido	Mapeado al WM8731 vía I2C
Tiempo de inicio < 10 s	Cumplido	Boot automático U-Boot + Linux
Respuesta a botones < 200 ms	Cumplido	Polling cada $\sim 31 \mu\text{s}/\text{frame}$

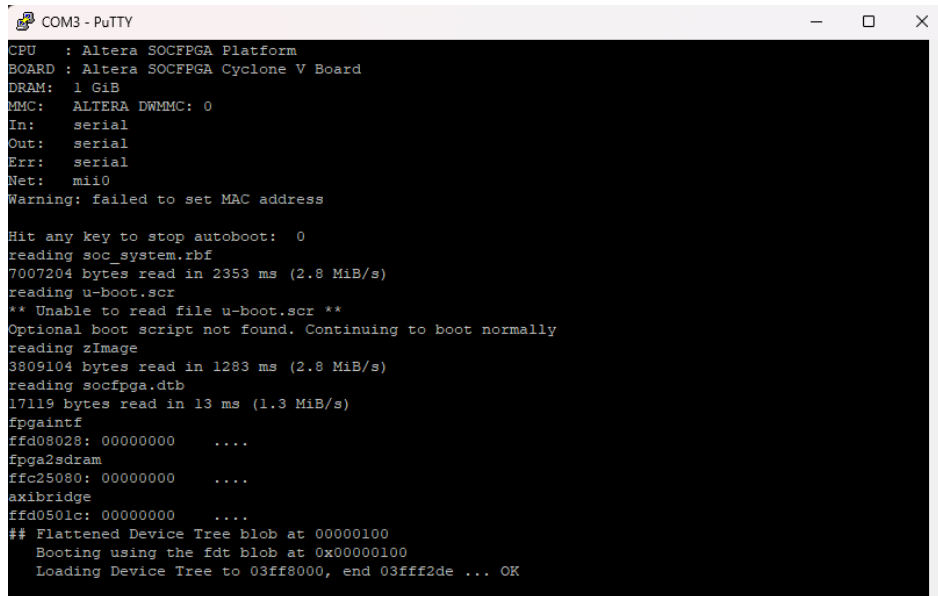
Cuadro 8: Estado de cumplimiento de todos los requerimientos

5.2. Pruebas realizadas

5.2.1. Arranque automático del sistema

Durante la prueba se verificó que el tiempo aproximado de arranque completo fuera de alrededor de 10 segundos, considerando desde el encendido de la tarjeta hasta el inicio de la aplicación de reproducción. Para reducir el tiempo de espera, se dejó la configuración de arranque lo más mínima posible, evitando procesos innecesarios durante el inicio del sistema y manteniendo únicamente los servicios requeridos para la ejecución del reproductor.

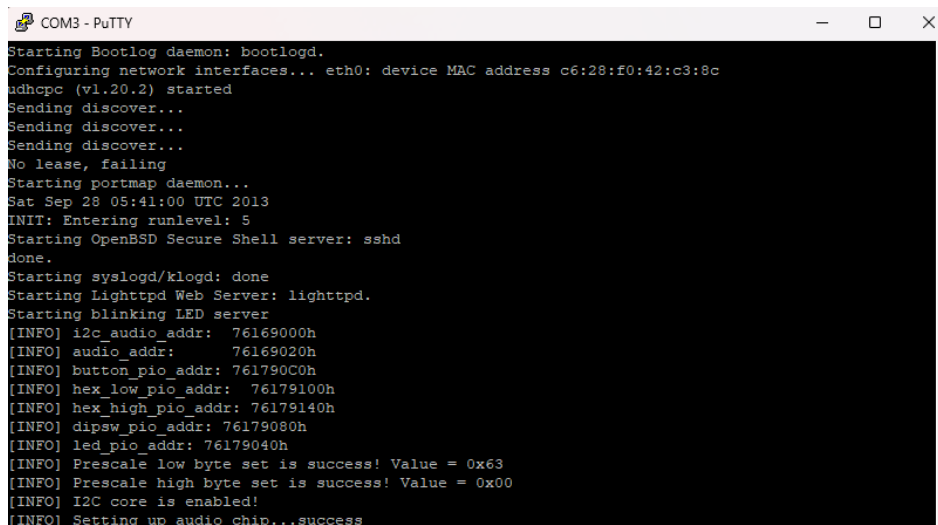
Además, se revisó que el proceso de arranque no requiriera interacción desde la consola serial ni comandos manuales posteriores al encendido. Esto permite que el sistema se comporte como un reproductor embebido autónomo, donde la tarjeta inicializa Linux, configura los periféricos necesarios y ejecuta la aplicación principal de manera automática.



```
COM3 - PuTTY
CPU : Altera SOCFPGA Platform
BOARD : Altera SOCFPGA Cyclone V Board
DRAM: 1 GiB
MMC: ALTERA DWMMC: 0
In: serial
Out: serial
Err: serial
Net: mii0
Warning: failed to set MAC address

Hit any key to stop autoboot: 0
reading soc_system.rbf
7007204 bytes read in 2353 ms (2.8 MiB/s)
reading u-boot.scr
** Unable to read file u-boot.scr **
Optional boot script not found. Continuing to boot normally
reading zImage
3809104 bytes read in 1283 ms (2.8 MiB/s)
reading socfpga.dtb
17119 bytes read in 13 ms (1.3 MiB/s)
fpgaintf
ffd08028: 00000000 ....
fpga2sdram
ffc25080: 00000000 ....
axibridge
ffd0501c: 00000000 ....
## Flattened Device Tree blob at 00000100
Booting using the fdt blob at 0x00000100
Loading Device Tree to 03ff8000, end 03fff2de ... OK
```

Figura 6: Consola serial (PuTTY) con Uboot con *soc_system.rbf*



```
COM3 - PuTTY
Starting Bootlog daemon: bootlogd.
Configuring network interfaces... eth0: device MAC address c6:28:f0:42:c3:8c
udhcpd (v1.20.2) started
Sending discover...
Sending discover...
Sending discover...
No lease, failing
Starting portmap daemon...
Sat Sep 28 05:41:00 UTC 2013
INIT: Entering runlevel: 5
Starting OpenBSD Secure Shell server: sshd
done.
Starting syslogd/klogd: done
Starting Lighttpd Web Server: lighttpd.
Starting blinking LED server
[INFO] i2c_audio_addr: 76169000h
[INFO] audio_addr: 76169020h
[INFO] button_pio_addr: 761790C0h
[INFO] hex_low_pio_addr: 76179100h
[INFO] hex_high_pio_addr: 76179140h
[INFO] dipsw_pio_addr: 76179080h
[INFO] led_pio_addr: 76179040h
[INFO] Prescale low byte set is success! Value = 0x63
[INFO] Prescale high byte set is success! Value = 0x00
[INFO] I2C core is enabled!
[INFO] Setting up audio chip...success
```

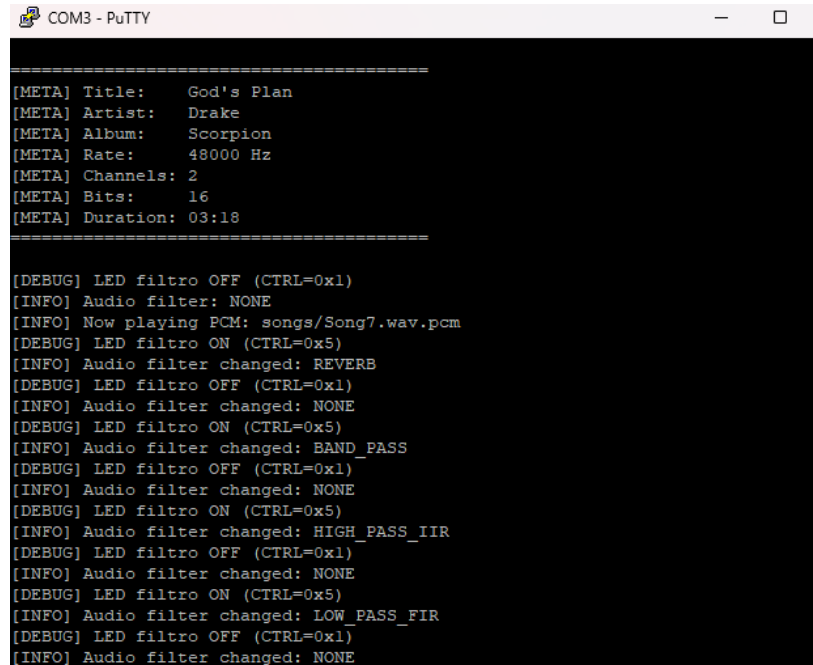
Figura 7: Consola serial (PuTTY) con daemon MyPlayer

5.2.2. Pruebas de filtros de audio

Cada filtro se verificó cualitativamente durante reproducción de canciones de diferentes géneros:

- **Pasa-bajos FIR:** sonido perceptiblemente apagado. Efecto muy claro en música con presencia de platillos e instrumentos de alta frecuencia.
- **Pasa-altos IIR:** graves atenuados con presencia de medios y agudos. Efecto consistente aunque más sutil que el FIR.

- **Reverberación:** eco perceptible con delay de ~ 150 ms. Muy notorio en voces y guitarras acústicas; el efecto es inmediatamente audible.
- **Pasa-banda biquad:** efecto tipo “radio AM/teléfono” muy pronunciado, concentrando el sonido alrededor de 1 kHz.
- **Cambio entre filtros:** transiciones sin artefactos audibles (clicks o transitorios), gracias a la reinicialización del estado interno del filtro anterior antes de activar el nuevo.



```

COM3 - PuTTY

=====
[META] Title:      God's Plan
[META] Artist:     Drake
[META] Album:      Scorpion
[META] Rate:       48000 Hz
[META] Channels:    2
[META] Bits:       16
[META] Duration:   03:18
=====

[DEBUG] LED filtro OFF (CTRL=0x1)
[INFO] Audio filter: NONE
[INFO] Now playing PCM: songs/Song7.wav.pcm
[DEBUG] LED filtro ON (CTRL=0x5)
[INFO] Audio filter changed: REVERB
[DEBUG] LED filtro OFF (CTRL=0x1)
[INFO] Audio filter changed: NONE
[DEBUG] LED filtro ON (CTRL=0x5)
[INFO] Audio filter changed: BAND_PASS
[DEBUG] LED filtro OFF (CTRL=0x1)
[INFO] Audio filter changed: NONE
[DEBUG] LED filtro ON (CTRL=0x5)
[INFO] Audio filter changed: HIGH_PASS_IIR
[DEBUG] LED filtro OFF (CTRL=0x1)
[INFO] Audio filter changed: NONE
[DEBUG] LED filtro ON (CTRL=0x5)
[INFO] Audio filter changed: LOW_PASS_FIR
[DEBUG] LED filtro OFF (CTRL=0x1)
[INFO] Audio filter changed: NONE

```

Figura 8: Consola serial (PuTTY) con los filtros en ejecución

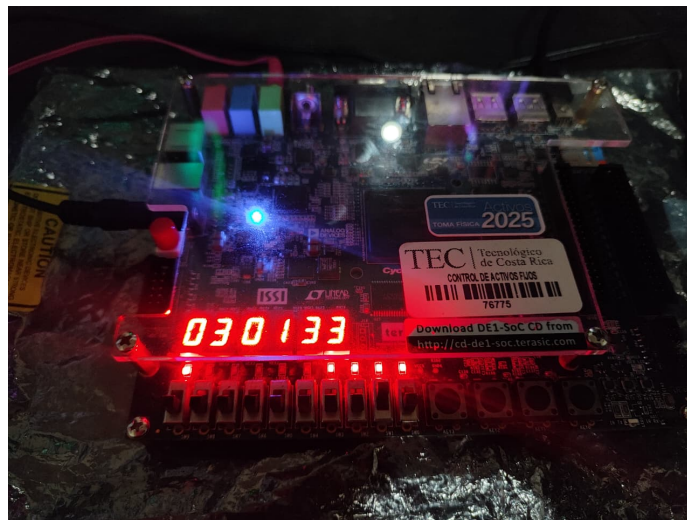


Figura 9: Led indicador de filtro activo: LED[0]

5.2.3. Pruebas de metadatos WAV

Se verificó el parser en tres escenarios distintos:

1. **WAV con chunk LIST/INFO embebido** (generado con `ffmpeg -metadata title=... -metadata artist=...`): extrae título, artista y álbum correctamente desde los sub-chunks INAM, IART e IPRD.
2. **WAV con nombre formato `Artista - Título.wav`**: fallback por nombre de archivo funciona correctamente, separando en el delimitador `` - ''.
3. **WAV con nombre genérico (`song1.wav`)**: muestra `song1` como título y `Unknown` como artista, comportamiento esperado y documentado.

```
=====
[META] Title:      Gravity
[META] Artist:     John Mayer
[META] Album:      Continuum
[META] Rate:       16000 Hz
[META] Channels:   2
[META] Bits:       8
[META] Duration:   03:59
=====
```

Figura 10: Metadatos obtenidos para cada canción mediante el .wav

5.2.4. Prueba de estabilidad

Se realizó una prueba de reproducción continua ejecutando el ciclo completo de la playlist durante varias iteraciones. El objetivo de esta prueba fue verificar que el sistema pudiera mantenerse en funcionamiento por un periodo prolongado sin presentar reinicios inesperados, bloqueos de la aplicación, fallos en la transición entre canciones o pérdida de calidad en la salida de audio. Durante la prueba se validó que, al finalizar una canción, el reproductor avanzara automáticamente a la siguiente pista de la lista y continuara la reproducción sin intervención del usuario. Además, se comprobó que los periféricos de salida, como los displays de 7 segmentos, el peakmeter, los LEDs de estado y la pantalla LCD, mantuvieran un comportamiento consistente durante la ejecución.

El sistema completó la reproducción continua de la playlist sin errores críticos. No se observaron reinicios del programa, bloqueos del HPS, fallos en la comunicación con los periféricos ni degradación progresiva del audio. Esto permite concluir que la integración entre la aplicación en C, los módulos de hardware en FPGA y los periféricos del sistema presenta un comportamiento estable durante la operación prolongada.

```
COM3 - PuTTY
[LCD][I2C WARN] NACK addr=0 data=1 byte=0xE9
[INFO] i2c_audio_addr: 76164000h
[INFO] audio_addr: 76164020h
[INFO] button_pio_addr: 761740C0h
[INFO] hex_low_pio_addr: 76174100h
[INFO] hex_high_pio_addr: 76174140h
[INFO] dipsw_pio_addr: 76174080h
[INFO] led_pio_addr: 76174040h
[INFO] Prescale low byte set is success! Value = 0x63
[INFO] Prescale high byte set is success! Value = 0x00
[INFO] I2C core is enabled!
[INFO] Setting up audio chip...success
[INFO] AUDIO InterfaceActive... success
[INFO] AUDIO DacEnableSoftMute... success
[INFO] AUDIO MicMute... success
[INFO] AUDIO LineInMute...
[INFO] AUDIO LineInMute...success
[INFO] AUDIO DacEnableSoftMute... success
[INFO] set Line-Out vol(60,60) success
[INFO] AUDIO DacEnableSoftMute... success
[INFO] AUDIO SetSampleRate... success
[INFO] AUDIO InterfaceActive... success
[INFO] Buttons initialized
[INFO] KEY0 = Play/Pause
[INFO] KEY1 = Stop
[INFO] KEY2 = Next
[INFO] KEY3 = Previous
[INFO] Switch filter control initialized
[INFO] SW0 = Reverb
[INFO] SW1 = Band Pass
[INFO] SW2 = High Pass
[INFO] SW3 = Low Pass
[INFO] SW0-SW3 OFF = No filter
[INFO] LED filter indicator initialized (via peakmeter module)
[INFO] LEDR[0] = Filter active indicator
[INFO] 7-segment display initialized

COM3 - PuTTY
[INFO] AUDIO SetSampleRate... success
[INFO] Playlist loaded
[INFO] Songs found: 10
[INFO] Song 1: songs/Song5.wav
[INFO] Song 2: songs/song2_meta.wav
[INFO] Song 3: songs/Song7.wav
[INFO] Song 4: songs/song1_meta.wav
[INFO] Song 5: songs/Song9.wav
[INFO] Song 6: songs/Song6.wav
[INFO] Song 7: songs/Song8.wav
[INFO] Song 8: songs/Song4.wav
[INFO] Song 9: songs/song3_meta.wav
[INFO] Song 10: songs/Song10.wav

[INFO] Controls:
KEY0: Play/Pause
KEY1: Stop
KEY2: Next
KEY3: Previous

[INFO] Selected song [1/10]: songs/Song5.wav
[INFO] PCM already exists: songs/Song5.wav.pcm
[INFO] Now playing: songs/Song5.wav
[INFO] Duration: 05:21
[INFO] set Line-Out vol(58,58) success
[INFO] Volumen: 16/31 (WM8731: 0x58)
[LCD] field=TITLE | line1="Titulo:" | line2="Californication"
```

Figura 11: Evidencia de inicialización del sistema.

5.2.5. Prueba de la LCD HD44780

Se verificó la secuencia de inicialización en modo 4-bit y la correcta visualización de título y artista en ambas filas. Se confirmó el truncamiento limpio de strings más largos de 16 caracteres sin desbordamiento de buffer, y que la pantalla no presenta caracteres erróneos por problemas de timing en los pulsos EN. También se verificó el correcto funcionamiento del desplazamiento de metadatos cuando la canción se encuentra en pausa, con frecuencia de actualización cada 300 ms.

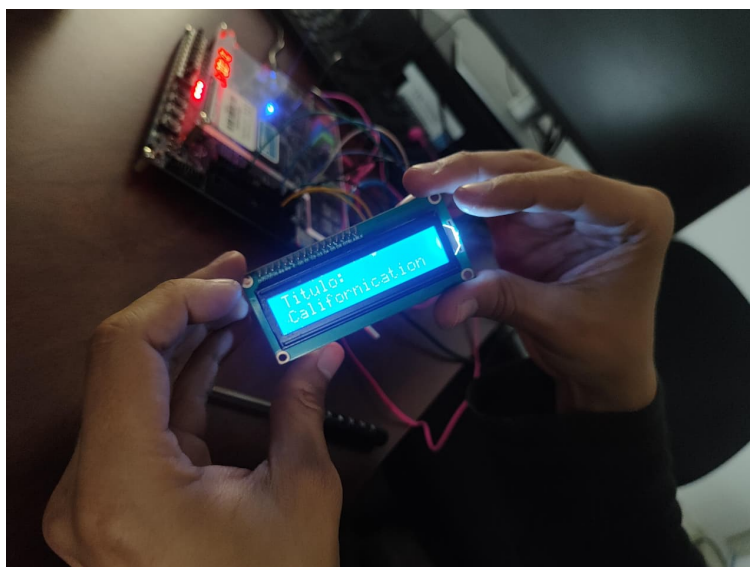


Figura 12: Demostración del funcionamiento de la LCD con metadatos (título)



Figura 13: Demostración del funcionamiento de la LCD con metadatos (duración)

5.3. Trade-offs observados

- **Filtros en SW vs HW:** La carga al procesador por el procesamiento DSP en software resultó completamente manejable. El FIFO del DAC nunca se vació durante la reproducción, lo que confirma que el ARM Cortex-A9 procesa las muestras con margen suficiente a 32 kHz.
- **LCD vía GPIO HPS:** El timing de `usleep()` resultó suficientemente preciso para el protocolo HD44780. En un sistema de tiempo real estricto esto podría ser problemático, pero para una aplicación de visualización de metadatos la variación es imperceptible.
- **Conversión WAV a PCM con FFmpeg:** Simplificó el código del reproductor notablemente. El único costo es el tiempo de conversión en la primera reproducción de cada canción, que oscila entre 3 y 10 segundos dependiendo de la duración del archivo.
- **Acceso directo sin HAL:** Mayor verbosidad en el código a cambio de control total y comprensión precisa del mapeo de memoria. Facilita la depuración ya que cada acceso a hardware es explícito y trazable.

6. Conclusiones

El desarrollo del Proyecto II permitió aplicar en la práctica los principios del co-diseño hardware/software en un sistema embebido real sobre la plataforma DE1-SoC. La solución final integró periféricos implementados en FPGA, acceso mediante memoria mapeada desde el HPS, procesamiento de audio en software y control físico del sistema mediante botones, switches, encoder, LEDs, displays de 7 segmentos y LCD.

- **DI1:** Se identificaron y formalizaron los requerimientos funcionales y no funcionales del sistema, incluyendo restricciones de rendimiento, latencia, arranque automático y acceso directo al hardware mediante MMIO.
- **DI2:** Se evaluaron alternativas de partición hardware/software para los componentes principales del sistema, justificando técnicamente la asignación de tareas al HPS o a la FPGA según criterios de latencia, flexibilidad, complejidad de implementación y disponibilidad de recursos.

- **DI3:** Se diseñó e implementó una solución completa con documentación del mapa de memoria, interfaz de módulos de hardware, arquitectura modular del software, módulo personalizado de peakmeter y control de LCD mediante un PIO bidireccional integrado en Platform Designer.
- **DI4:** Se validó el sistema mediante pruebas funcionales en hardware real, documentando resultados, evidencias de ejecución y trade-offs observados durante la integración del sistema.

El sistema final opera de forma estable y cumple con los requerimientos principales de la especificación. Se implementó reproducción de audio desde archivos PCM generados a partir de canciones WAV, control físico completo de reproducción, cuatro filtros de audio seleccionables, visualización del tiempo en formato ##:MM:SS, peakmeter en tiempo real, control de volumen mediante encoder, visualización de metadatos en consola y LCD, y arranque automático desde el encendido de la tarjeta.

En conjunto, el proyecto evidenció la importancia de una partición adecuada entre HPS y FPGA. Las tareas con requerimientos de interacción directa con periféricos y baja latencia se resolvieron mediante hardware o PIOs en FPGA, mientras que las tareas de mayor flexibilidad, como manejo de archivos, filtros de audio, metadata y control del flujo de reproducción, se implementaron en software sobre Linux embebido.

7. Referencias

- Terasic. (s.f.). *DE1-SoC Development and Education Kit*. Terasic Inc.
<https://www.terasic.com.tw/cgi-bin/page/archive.pl?No=836>
- Intel/Altera. (2018). *Cyclone V Hard Processor System Technical Reference Manual*.
- Intel/Altera. (2018). *Platform Designer User Guide*, Quartus Prime 18.1.
- Hitachi. (s.f.). *HD44780U Dot Matrix LCD Controller/Driver Datasheet*.
- Wolfson Microelectronics. (s.f.). *WM8731 Portable Internet Audio CODEC Datasheet*.
- Conventional Commits. <https://www.conventionalcommits.org/>
- Git Flow. <https://nvie.com/posts/a-successful-git-branching-model/>